Before writing a single line of code, I sat down and mapped out the different pages the app would need and thought through what information each page would have to display in order to give a user a good experience. That planning stage was important because it forced me to think about the full journey, from the moment someone opens the app to the moment they find something they actually want to buy before I get caught up in the implementation details.

The core concept behind EveryFit is straightforward: clothing shopping is frustrating when the results you see are not built for your body type. Someone who is petite should not have to scroll past dozens of tall-fit results to find what works for them, and vice versa. I wanted the app to quietly handle that filtering in the background, so the experience feels personalized from the start rather than something the user has to configure every single time they search.

To power the product catalog, I chose the ASOS API available through RapidAPI. ASOS has a large and well-organized inventory, which made it a natural choice. But what I discovered early on is that not all APIs that come from the same provider or even the same author on RapidAPI share the same data. That assumption turned out to be one of the first real technical challenges of the project.

When I was putting together my project proposal, I parsed two different ASOS APIs and assumed they would share the same product data since they came from the same source. That was a wrong assumption. What I found when I dug deeper is that one API had an extensive product catalog with images, names, prices, and URLs, while the other had detailed customer reviews but they were not talking about the same products. Their product IDs did not match up.

My original plan was to connect both APIs by matching product IDs, so I could show a product's image and details on one side and pull in real customer reviews from the other. Once I realized those IDs did not align, I had to make a decision about which API to build around. I ended up choosing the review-focused API, even though it meant working within a different product catalog. The review data was genuinely useful for what I wanted to build an AI-powered insight feature, and the catalog it came with was still large enough to make the shopping experience feel real. Either one would have been a fine foundation for the clothing data itself.

This was an early lesson in not assuming that APIs from the same ecosystem speak the same language. Reading the actual JSON responses side by side was what finally made the mismatch obvious, and it was a good reminder that the planning and parsing stage is not something you can skip.

Building the Search and Fit Filter

Once I settled on the API, the next step was making sure it was actually pulling the right information product names, images, prices, and URLs and displaying them cleanly in a grid. Getting the data to flow into the UI widgets was honestly the most straightforward part of the project. The interesting challenge came when I tried to implement the fit filtering.

The ASOS API does not expose a dedicated sizing category field. There is no flag that says 'this item is petite' or 'this item is tall.' What you have is a product description string, and that is where I had to get creative. My approach was to scan the description text for the keywords 'petite' or 'tall.' If neither word appeared, I treated the item as regular sizing. This mirrors what ASOS itself does: standard-fit garments typically do not include a sizing qualifier in their title, while petite and tall items almost always do. So the absence of either keyword is itself meaningful information.

Regular sizing was the trickiest case to handle because I had to decide what the absence of a label actually means in practice. After thinking it through, I felt confident that assuming regular when neither 'petite' nor 'tall' appears in the title was a reasonable and defensible heuristic. It may not be perfect for every edge case, but it aligns with how the retailer itself categorizes its inventory.

I also briefly explored whether review language could help with fit detection specifically, whether items that reviewers consistently described as 'runs small' could be flagged as petite-friendly. I decided against implementing this, due time constraints and also in terms of how the AI evaluates what words or an equation to determine, I think if I do want to further this project in my time this will be something that I look into, as I do plan to use the project I made during this semester extend them further over the summer, with a bit more time to explore various methods to do this.

The Price Slider and Search Controls

Beyond fit, I added a price range slider and a result limit dropdown so users have some control over how they search. The price filter is applied client-side after the API responds, meaning the app fetches up to the requested limit and then narrows the visible results based on where the slider sits. The fit preference a user selects during onboarding is saved locally and automatically populates the fit dropdown every time they return to the search page, so they never have to set it again unless they want to change it.

Onboarding and Saving Preferences

I wanted the app to feel like it already knows the user by the time they get to searching. To make that happen, the very first thing EveryFit does is walk the user through a short onboarding flow. The Welcome Page is intentionally simple just the app name and a single button to begin. From there, the Preference Form asks one question: how do clothes usually fit you? The options are Petite, Regular, and Tall.

When the user makes their selection and taps save, that choice gets stored locally using the SharedPreferences package. SharedPreferences is a lightweight key-value store that persists data across app sessions without requiring a database. The next time the user opens the app, the saved value is read during the search page's initialization and automatically applied to the fit

filter. The onboarding screens are replaced rather than pushed onto the navigation stack, so the user cannot accidentally navigate back to them after completing setup.

Learning how to implement this kind of persistent preference was genuinely interesting. I had a clear picture of what I wanted it to do to fill in the filter automatically but figuring out the right Flutter pattern to make that happen cleanly took some research. I used AI to help me understand the concept and point me toward the right tools, but I made sure to work through the implementation myself so I actually understood what was happening under the hood.

The Product Detail Page and Getting to Cart

Developing the product detail view and the cart flow was the part of the project that took the most time and required the most reading and research. When a user taps on any item in the search grid, they are taken to a detail page that shows a large image of the product, the price, the brand name, an AI-generated review summary, and an Add to Cart button. Getting all of those pieces to talk to each other cleanly especially with the async nature of the AI call was a real exercise in Flutter state management.

The detail page starts loading the Gemini review summary the moment it mounts. Rather than waiting for that to finish before showing anything to the user, the page renders immediately with a progress indicator in the AI insight panel and replaces it with the actual summary once the async work is done. This pattern keeps the interface feeling responsive rather than frozen. I used Stack Overflow and AI guidance to understand the right way to structure this in Flutter not to get the code written for me, but to understand the concepts well enough to implement them myself. I think that distinction matters, especially for something like async state handling that comes up constantly in real app development.

The cart itself uses a global list that any part of the app can add to or read from. When the user taps Add to Cart on the detail page, the product's data map gets appended to that list and a snackbar appears confirming the action with a shortcut link to the cart. The approach is intentionally simple for a prototype, in doing some research I did see other methods real production app use that were formal statement solutions which was interesting but the method to ensure I was not going overboard with scope.

The Swipe Actions in Cart

One of the features I am happiest with is how the cart handles item interactions. Each item card supports two swipe directions. Swiping left to right opens the product directly on the ASOS website in the device's browser, using the url_launcher package to handle the deep link. Swiping right to left removes the item from the cart with an animation. I wanted to make both actions feel natural and discoverable, so there are color cues and icons on each side to hint at what each direction does.

Integrating Gemini AI API

Connecting the Gemini API was the most challenging and also the most rewarding part of this project. I had not realized before starting this that you could write a prompt directly inside code and have it shape how a model responds that felt genuinely cool to discover. The idea is that for each product, the app fetches whatever customer reviews exist from the ASOS API, passes them into a structured prompt, and asks Gemini to summarize them in plain English the way a fashion stylist would.

Getting the prompt right took real thought. I needed the output to be useful but also to render cleanly in the app without any post-processing. That meant telling the model explicitly: write exactly two sentences, focus on quality, comfort, and sizing, do not use bullet points, do not use bold text, and if any data field says N/A just ignore it and work from the written review text instead. Those constraints were not just stylistic preferences, they were functional requirements, because the output was going straight into a plain Flutter Text widget.

One concern I had throughout testing was token consumption. Every time a user opens a product detail page, a live call goes out to the Gemini API, and I was not sure how quickly costs would add up. I made two practical decisions to manage this. First, I chose Gemini 1.5 Flash rather than a more powerful but more expensive model variant, Flash is designed for exactly this kind of lightweight, structured summarization task. Second, the two-sentence output constraint naturally keeps token counts low. The way the async chain is structured also helps: the app has to finish fetching reviews from the ASOS API before it starts the Gemini call, which means there is a natural delay between requests when a user browses through products quickly. That is not a designed rate limiter, but it works like one.

When the review fetch returns nothing either because the product has no reviews yet or because the API call fails the app falls back to a friendly default message rather than showing an error or crashing. The same fallback fires if the Gemini call itself encounters an error. There is also a mounted check before every setState() call in the async chain, which prevents a common Flutter crash that happens when the user navigates away from a page before an async operation finishes.

Reflections and What I Would Do Differently

Looking back at the full arc of this project, the things that took the longest were not the ones I expected. The UI grid and card layouts came together quickly, I do wish I had more time to make it look aesthetically better. The hard parts were the ones that involved understanding how different systems interact two APIs that turned out not to share data, an AI model that needed careful prompting to produce usable output, and async state in Flutter that required a real understanding of lifecycles to get right.

One feature I thought about but could not implement in time was pulling the actual size chart from ASOS product pages and using numeric measurements to power the fit recommendations. That would have been more accurate than scanning for keywords in the title, but it would have required either web scraping or a different API endpoint, and the time was not there. Similarly, I

considered whether review language about sizing could be used to make smarter recommendations, but I concluded that the signal was too noisy to be trustworthy without more analysis of the reviews.

If I were building a second version of this app, the first thing I would change is how API keys are handled. Right now they are stored as string constants in the source code, which is fine for a class project but would be a real security issue in a production environment. A proper implementation would use environment variables or a backend proxy so the keys are never exposed in the client. I would also add persistence to the cart, use a formal state management solution instead of a global list, and look into whether the ASOS product search endpoint could be used alongside the review endpoint to give users a broader catalog while still surfacing AI-powered insights.

Overall, this project pushed me to learn things I genuinely did not know going in how to wire an AI model into a mobile app or how SharedPreferences works, and how to handle async operations in Flutter without the UI locking up or crashing.